

CRUD Persistente

1. Introducción al CRUD Persistente

En el desarrollo backend actual, implementar un CRUD (Create, Read, Update, Delete) conectado a una base de datos representa el corazón de cualquier aplicación. Un CRUD persistente resuelve tres problemas históricos:

- Pérdida de datos al reiniciar el servidor.
- Inconsistencia de información por falta de validaciones.
- Exposición de datos sensibles al cliente.

El objetivo no es solo "guardar datos", sino construir desde el inicio una arquitectura robusta, segura y mantenible.

2. El Flujo Central de Datos

Para garantizar la integridad de la aplicación, toda operación de escritura o lectura debe seguir un flujo estricto y unidireccional.

Este flujo consta de cuatro etapas obligatorias:

- **Request:** Recepción de la petición.
- **Validación:** Control de calidad de los datos.
- **Commit:** Persistencia segura.
- **Response Model:** Salida controlada.

Saltarse una de estas etapas no es un atajo; es una falla de seguridad.

2.1 Request (La Petición)

Es el punto de entrada de la aplicación. El cliente (frontend o aplicación móvil) envía una solicitud HTTP con información. Características principales:

- Utiliza verbos HTTP (POST para crear, GET para leer, PUT/PATCH para actualizar, DELETE para borrar).
- Transporta datos en el *body* (generalmente en formato JSON).
- Debe ser tratada como información "no confiable" por defecto.

2.2 Validación

Antes de tocar la lógica de negocio o la base de datos, la petición debe ser validada.

Principios fundamentales:

- Filtro estricto de tipos de datos (ej: el precio debe ser un número, no un texto).
- Validación de reglas de negocio (ej: el stock no puede ser negativo).
- Rechazo temprano: si la validación falla, se devuelve un error 400 (Bad Request) inmediatamente.
- Uso de esquemas (DTOs - Data Transfer Objects) para definir el contrato de entrada.

2.3 Commit (Persistencia)

Es el momento donde los datos validados interactúan con la base de datos de forma persistente.

Ventajas del uso de transacciones:

- **Atomicidad:** O se guardan todos los datos correctamente, o no se guarda ninguno.
- Si ocurre un error durante la inserción, se realiza un *Rollback* (se deshacen los cambios).
- Si todo es correcto, se realiza el *Commit* (se confirman los cambios de forma permanente en el disco).

2.4 Response Model (Modelo de Respuesta)

Nunca se debe devolver el registro crudo de la base de datos directamente al cliente. El Response Model actúa como un filtro de salida. Ventajas:

- Oculta información sensible (contraseñas, tokens, IDs internos).
- Formatea los datos según lo que el cliente realmente necesita.
- Garantiza que la estructura de la respuesta sea predecible y tipada.

3. Estructura del Proyecto

Para mantener este flujo ordenado, el proyecto backend debe dividirse en responsabilidades claras:

nombre-proyecto/

|

|— main.py

|— database.py

|

|— app/

|— routers/ (Maneja los Requests)

|— schemas/ (Maneja la Validación y Response Models)

|— models/ (Maneja la estructura de la Base de Datos)

|— service/ (Lógica de negocio y control de transacciones)

3.1 Archivos clave

routers/ Define los endpoints. Recibe el *Request* y devuelve el *Response Model*.

schemas/ Define los contratos de entrada y salida (Validación). Nunca se conectan a la base de datos.

models/ Representan las tablas de la base de datos.

service/ Contiene la lógica de negocio. Es responsable de manejar las transacciones (add, commit, refresh, delete) y decidir cuándo confirmar o revertir cambios.

4. Implementación del Flujo (Ejemplo Paso a Paso)

Tomemos como ejemplo la creación de un producto para una tienda virtual.

Paso 1: Validación y Response Model (schemas)

Definimos qué entra y qué sale.

Esquema de Entrada (Validación)

```
class ProductoCreate(BaseModel):
```

```
    nombre: str
```

```
    precio: float
```

```
    stock: int
```

Esquema de Salida (Response Model)

```
class ProductoResponse(BaseModel):
```

```
    id: int
```

```
    nombre: str
```

```
    precio: float
```

Nota: El Response Model no devuelve el stock, protegiendo información interna de la tienda.

Paso 2: El Modelo de Base de Datos (models)

Definimos la tabla persistente.

```
from typing import Optional
```

```
from sqlmodel import SQLModel, Field
```

```
class Producto(SQLModel, table=True):
```

```
    id: Optional[int] = Field(default=None, primary_key=True)
```

nombre: str

precio: float

stock: int

Paso 3: El Service (Lógica de Negocio y Transacción)

El service es responsable de controlar la transacción completa.

Recibe los datos ya validados, crea la entidad de base de datos, ejecuta el commit y refresca el objeto antes de devolverlo.

```
# service/producto_service.py

from sqlmodel import Session

from app.models.producto import Producto

from app.schemas.producto import ProductoCreate


def crear_producto(session: Session, data: ProductoCreate) -> Producto:

    # 1. Transformamos el DTO (ProductoCreate) en modelo de tabla

    nuevo_producto = Producto(**data.model_dump())

    # 2. Agregamos la entidad a la sesión

    session.add(nuevo_producto)

    # 3. Confirmamos la transacción (persistencia real en la BD)

    session.commit()
```

4. Refrescamos para obtener valores generados (ej: ID)

```
session.refresh(nuevo_producto)
```

```
return nuevo_producto
```

Paso 4: El Request (routers)

Unimos todo en el endpoint.

Python

```
@app.post("/productos/", response_model=ProductoResponse)
```

```
def endpoint_crear_producto(producto: ProductoCreate, db: Session):
```

```
    # El framework ya hizo el Request y la Validación gracias a ProductoCreate
```

```
    # Delegamos la lógica y la transacción al service
```

```
    producto_creado = crear_producto(db, producto)
```

```
    # El framework filtra la salida gracias al response_model
```

```
    return producto_creado
```

5. Buenas Prácticas Iniciales

- Separar siempre los esquemas de entrada de los de salida.
- Usar transacciones (`commit` y `rollback`) para evitar datos corruptos.
- Delegar la interacción de la base de datos a la capa service.
- Mantener los endpoints (routers) lo más limpios y cortos posible.
- Tipar absolutamente todas las entradas y salidas.

6. Errores Comunes

- ✗ Recibir un JSON genérico sin validarlo.
- ✗ Hacer un `.commit()` directamente en el archivo del endpoint (router).
- ✗ Devolver el modelo de base de datos completo al cliente exponiendo datos.
- ✗ Confundir un Modelo de Base de Datos (ORM) con un Esquema de Validación.

7. Conclusión

El flujo **Request** → **Validación** → **Service (Transacción)** → **Response Model** no es una sugerencia; es el estándar de la industria. Permite:

- Seguridad desde la puerta de entrada.
- Manejo predecible de errores.
- Persistencia confiable.
- Código escalable y profesional.

Una arquitectura sólida en el backend evita fallas críticas en producción. Primero la integridad de los datos. Después las funcionalidades extra.